

Week 4 — GUI & Problem-Solving Agent

Streamlit application (`app.py`) covering Lab Tasks 1–3. Course: AI Lab · CLO-2

1. What the application is

A single Streamlit script, `app.py`, containing all three graded tasks. A radio control in the sidebar selects which task is shown, so one deployed URL serves the whole lab.

Sidebar option	Task	What it does
1: EDA Interface	Task 1	Upload a CSV, inspect its metadata, plot any attribute.
2: Cloud Deployment Agent	Task 2	Plans the cloud deployment sequence and its total time.
3: Course Registration Agent	Task 3	Plans the course-registration sequence and its total cost.

Running it

```
pip install -r requirements.txt
streamlit run app.py          # opens http://localhost:8501
```

Dependencies: `streamlit`, `pandas`, `numpy`, `matplotlib`, `seaborn`, `scikit-learn`. Test dataset: `Titanic-Dataset.csv`.

2. How Streamlit executes this file

Streamlit re-runs the *entire script top to bottom* on every interaction — every upload, dropdown change, or button click. The code is written with that in mind:

- `st.set_page_config(...)` is the first Streamlit call, as required.
- The sidebar radio returns a plain string; an `if / elif / else` on that string means only the selected task's code runs.
- The search algorithm runs only inside `if st.button(...)`, so no computation happens on an idle rerun (Key Point 2 of the manual).
- `st.stop()` ends the rerun early when there is nothing to show yet — no file uploaded, or an unparseable file.

3. Task 1 — Exploratory Data Analysis interface

3.1 Dataset ingestion & validation

```
uploaded_file = st.sidebar.file_uploader("Upload CSV File for Analysis", type="csv")
if uploaded_file is None:
    st.info("Please upload a CSV file to start EDA.")
    st.stop()
try:
    df = pd.read_csv(uploaded_file)
except Exception as e:
    st.error(f"Could not parse this file as CSV: {e}")
    st.stop()
if df.empty or df.columns.empty:
    st.error("The uploaded CSV has no data.")
    st.stop()
```

`type="csv"` restricts the picker to CSV files, and the `try/except` plus the emptiness check are the format validation the task asks for: a renamed or corrupt file produces a readable error instead of a Python traceback in the browser.

3.2 Metadata panel

Requirement	Code
First 5 rows	<code>st.dataframe(df.head())</code>
Dimensional properties	<code>df.shape[0]</code> rows × <code>df.shape[1]</code> columns
Column-wise data types	<code>df.dtypes.astype(str).to_frame("Data Type")</code>
Missing values per attribute	<code>df.isna().sum()</code> and <code>df.isna().mean()*100</code> (count and percentage, side by side)
Statistical summary	<code>df.select_dtypes("number").agg(["mean", "median", "min", "max"]).T</code>

`.astype(str)` on the dtypes is needed because Streamlit's table renderer cannot serialise raw NumPy dtype objects. `select_dtypes` isolates the numerical columns, and `.T` transposes the result so each column of the dataset becomes a row of the summary — the same orientation as the expected output in the task sheet. If the file has no numerical columns at all, a short message replaces the table rather than an empty frame.

3.3 Attribute selection & automated typing

```
column = st.sidebar.selectbox("Select Attribute for Visualization", df.columns)
series = df[column].dropna()
column_type = "Numerical" if pd.api.types.is_numeric_dtype(series) else "Categorical"
```

The dropdown is populated straight from `df.columns`, so it adapts to any uploaded dataset. Typing is decided by pandas itself through `is_numeric_dtype` rather than by a hand-written list of type names, which keeps integers, floats and booleans classified correctly. `dropna()` is applied once here so both the statistics and the plot ignore missing entries (important for Titanic's Age, which is ~20% missing).

3.4 Conditional visualisation

```
fig, ax = plt.subplots(figsize=(8, 4))
if column_type == "Numerical":
    sns.histplot(series, kde=True, ax=ax)
    ax.set(title=f"Histogram of {column}", xlabel=column, ylabel="Frequency")
else:
    counts = series.value_counts().head(20)
    sns.barplot(x=counts.index.astype(str), y=counts.values, ax=ax)
    ax.set(title=f"Frequency of {column}", xlabel=column, ylabel="Count")
    ax.tick_params(axis="x", rotation=45)
    for bar, n in zip(ax.patches, counts.values):
        ax.annotate(f"{n / len(series) * 100:.1f}%", ...)
st.pyplot(fig)
```

- **Numerical branch** — a seaborn histogram with a KDE overlay, titled and axis-labelled as specified.
- **Categorical branch** — `value_counts()` gives the frequency of each unique value; the bar chart is annotated with each bar's percentage share (the optional requirement). Labels are rotated 45° so long category names stay readable.
- `.head(20)` caps the chart at the twenty most frequent values. A free-text column such as `Name` has 891 unique values and would otherwise render an unreadable wall of bars.
- Because Streamlit reruns the script, changing the dropdown rebuilds the figure automatically — that is the "dynamic update" requirement; no callback code is needed.

The layout requirement is met by placing both controls (`st.sidebar.file_uploader`, `st.sidebar.selectbox`) in the sidebar, with preview and metadata at the top of the main area and the visualization canvas below it.

4. Tasks 2 & 3 — the problem-solving agents

Both tasks are the same problem: a state-space graph with weighted transitions, a start state, a goal state, and a request for the execution sequence plus its total path cost. They therefore share one search function and one renderer, and differ only in the data passed in.

4.1 The search — `uniform_cost_search`

```
def uniform_cost_search(graph, start, goal):
    frontier = [(0, start, [start])]
    visited = set()
    while frontier:
        cost, node, path = heapq.heappop(frontier)
        if node == goal:
            return path, cost
        if node in visited:
            continue
        visited.add(node)
        for nxt, step in graph.get(node, []):
            if nxt not in visited:
                heapq.heappush(frontier, (cost + step, nxt, path + [nxt]))
    return None, None
```

Element	Role in the agent
<code>graph</code>	Transition model: an adjacency list mapping each state to <code>(next_state, cost)</code> pairs.
<code>frontier</code>	A min-heap ordered by accumulated cost, so the cheapest partial plan is always expanded first.
<code>visited</code>	Prevents revisiting a completed step — an explicit requirement of Task 3 and what keeps the search terminating on cyclic graphs.
<code>node == goal</code>	The goal test. Reaching it returns the full action sequence and its accumulated path cost.
<code>return None, None</code>	The frontier emptied without reaching the goal: no plan exists.

Uniform-cost search is used rather than plain BFS because the edges carry unequal weights (2, 4, 3, 5, 2, 1 minutes in Task 2); BFS would return the plan with the fewest steps, not the cheapest one. On these two linear workflows both happen to agree, but the algorithm stays correct if an alternative or shortcut route is later added to either graph.

4.2 The renderer — `render_agent`

One function draws the whole agent page for either task:

- **State space** — a table of every state and its meaning, generated from the `states` dictionary.
- **Transition model** — every edge as *From* → *Action* → *To* with its cost, built by flattening the adjacency list.
- **Run Planning Agent** button — gates the search, then prints the sequence `S0 → S1 → ... → goal`, a step-by-step table pairing each transition with its action name and cost, the total via `st.metric`, and the goal test result. If no plan is found it shows an error instead.

The step table is built by zipping the path with itself offset by one (`zip(path, path[1:])`), which turns the list of states into the list of transitions between them.

4.3 Task 2 — cloud resource deployment

Initial state `S0` (request received) → goal `S6` (deployment completed). Actions: allocate resources, create VM, install dependencies, deploy AI model, verify deployment. Costs are execution times in minutes: 2 + 4 + 3 + 5 + 2 + 1, so the agent reports a **17-minute** optimal plan, matching the state-space diagram in the task sheet.

4.4 Task 3 — university course registration

Initial state `S0` (student login) → goal `S5` (enrollment confirmed). Actions: authenticate student, verify prerequisites, select courses, verify fee status, confirm enrollment. Each step has a processing cost, totalling **7 units** — again matching the diagram. The `visited` set satisfies the "prevent revisiting completed registration steps" requirement.

5. Verification

The search was checked against four cases: both lab graphs (asserting 17 minutes and 7 units with the exact expected sequences), a graph where a longer-in-steps route is cheaper (confirming it returns the cheaper plan,

not the shorter one), and an unreachable goal (confirming (None, None)). The app was then launched headless and served HTTP 200 with no errors in the log.

6. Files

File	Purpose
app.py	The application — all three tasks.
requirements.txt	Dependencies for deployment.
Titanic-Dataset.csv	Test dataset for Task 1.
README.pdf	This document.